

Projekt-Protokoll — vergabe-intel-mcp

Projekt: Enterprise-MCP-Server „Vergabe-Intelligence“
Datum: 25.06.2026 · **Autor:** Andi (Awees67) · **Sektor:** Public & Infrastructure / Energy

1. Ausgangslage / Problem

Große Infrastruktur-/Energie-Organisationen müssen laufend **zehntausende öffentliche Ausschreibungen** sichten, um relevante Aufträge nicht zu verpassen — heute teils manuell über mehrere Portale. Folgen: verpasste Aufträge (Millionen-ROI), keine konsistente Priorisierung, kein Marktüberblick, mehrere Personentage Aufwand pro Woche.

2. Ziel

Ein **Enterprise-tauglicher, skalierbarer** MCP-Server, der einem KI-Agenten Werkzeuge gibt, um über **echte** Ausschreibungsdaten zu **suchen, aggregieren und nach Relevanz zu priorisieren** — mit Sicherheit, Pagination und Datenbank-Fundament.

3. Vorgehen (chronologisch)

1. **Discovery** — Enterprise-Pain, Ist-Prozess, Erfolgskriterien festgehalten (`01-discovery.md`). 2. **Architektur-Entscheid** — DB-gestützt, Pagination, Sicherheit, Trace-Logging (`02-architektur.md`). 3. **Umgebung geprüft** — Node `v24.16.0` ; eingebaute DB `node:sqlite` verfügbar. 4. **Datenmodell** — Tabelle `tenders` + 5 Indizes. 5. **Echter Ingest** — `ingest.mjs` holt **echte** Ausschreibungen aus der offiziellen **EU-TED-API** (`api.ted.europa.eu` , offen, ohne Key), mappt Mehrsprachigkeit → Deutsch, CPV → Sektor, ISO3- → ISO2-Land. **Ergebnis: 2.507 echte Ausschreibungen** (AT+DE, 2026; in der API **60.000+** verfügbar). 6. **Abfrage-Schicht** (`db.mjs`) — DB **read-only, parametrisierte** Abfragen, Whitelist für Gruppierungen. 7. **MCP-Server** (`vergabe-intel-mcp.mjs`) — 4 Werkzeuge, `zod` -Validierung, Pagination (Limit max 100), Trace-Logging, strukturierte Fehler. 8. **Smoke-Test** — Handshake + Werkzeug-Aufrufe gegen die **echte** DB mit Zeitmessung.

4. Datenquelle

EU TED — Tenders Electronic Daily, die offizielle Ausschreibungsdatenbank der EU. Endpoint `POST https://api.ted.europa.eu/v3/notices/search` (offen, ohne Key). Abruf paginiert (100/Seite), höflich gedrosselt. Dedup über Primärschlüssel (Publication-Number).

5. Umgesetzte Werkzeuge

Werkzeug	Aufgabe
<code>vergabe_suche</code>	gefilterte, paginierte Suche (Stichwort, Land, Sektor, Wertspanne, nur offene)
<code>vergabe_details</code>	eine Ausschreibung vollständig
<code>vergabe_statistik</code>	Aggregation nach Land / Sektor / Status (Anzahl + Gesamtwert)
<code>vergabe_match</code>	Relevanz-Scoring gegen ein Organisationsprofil (Score + Begründung)

6. Test-Protokoll (gemessen, gegen 2.507 ECHTE Datensätze)

Aufruf	Ergebnis (echte Daten)	Dauer
<code>initialize</code> + <code>tools/list</code>	OK, 4 Werkzeuge	—
<code>vergabe_suche</code> (Energie, AT, offen)	u. a. <i>Notstrom Uni Graz, Fernwärme NÖ</i>	~5 ms
<code>vergabe_statistik</code> (Sektor)	Bau 1.014 (€2,3 Mrd), Beratung 526, IT 175 ...	~12 ms
<code>vergabe_match</code> (Transport, AT, >1 Mio €)	<i>Güterbeförderung Stadt Wien €5,8 Mio, Gleisbau ÖBB €1 Mio</i>	~2 ms

Beispiel-Ergebnis `vergabe_match` (echt):

1. [Score 80] 434091-2026 – Österreich – Güterbeförderung für den Straßenbau 2026 - RV
Magistrat der Stadt Wien - MA 28 (AT) · Transport · €5.800.529
Treffergründe: Sektor Transport, Land AT, Wert über Schwelle

7. Sicherheits-Check (3-fach)

Prüfpunkt	Umsetzung
Sicherheitslücken	DB <code>read-only</code> · <code>parametrisierte SQL</code> · <code>zod</code> -Validierung · Whitelist für Gruppierung
Pagination	jedes Such-Tool: <code>limit</code> max 100 + <code>offset</code> + <code>total</code>
DB-Sicherheit	Least Privilege (read-only), keine dynamische SQL aus Nutzertext

8. Status

Fertig, getestet, dokumentiert — mit echten Daten. Läuft lokal über stdio. Die DB ist mit `node ingest.mjs` jederzeit aus der Live-TED-API neu befüllbar.

9. Nächste Schritte

- Aufnahme ins GitHub-Portfolio (`mcp-portfolio`).
- Produktion: zusätzlich **Kunden-ERP** als Quelle, **HTTP-Transport + Token-Auth**, gehostete DB (Postgres), Monitoring/Alerting/Audit-Log — bei **gleichem Werkzeug-Code**.